Workgroup: Network Management Operations
Internet-Draft: draft-cui-nmop-auto-test-00
Published: 5 July 2026
Intended Status: Informational
Expires: 6 January 2027
Authors:
Y. Cui
Tsinghua University
Y. Wei
Tsinghua University
K. Chi
Tsinghua University
X. Xie
Tsinghua University
S. Prabhu
Nokia

A Framework for AI-Assisted Network Protocol Testing from Specifications

Abstract

Network protocol testing is essential for validating that implementations conform to their specifications. Traditional testing approaches rely heavily on manual effort or protocol-specific models that are expensive to build and difficult to reuse as specifications evolve and new protocols emerge.

This document describes a framework for AI-assisted network protocol testing that decomposes the testing workflow into six stages: structured protocol representation, coverage scoping, test case generation, executable artifact generation, test execution, and feedback-based refinement. The framework emphasizes explicit stage boundaries and reviewable intermediate outputs, keeping the workflow auditable and traceable to the specification text.

The document discusses the design motivations and trade-offs behind the framework, presents examples from routing protocol testing, and identifies operational considerations and open issues for applying the framework in test environments.

The RFC Editor will remove this note

About This Document

This note is to be removed before publishing as an RFC.

The latest revision of this draft can be found at https://datatracker.ietf.org/doc/draft-cui-nmop-auto-test/. Status information for this document may be found at https://datatracker.ietf.org/doc/draft-cui-nmop-auto-test/.

Discussion of this document takes place on the NMOP Working Group mailing list (mailto:nmop@ietf.org), which is archived at https://mailarchive.ietf.org/arch/browse/nmop/. Subscribe at https://www.ietf.org/mailman/listinfo/nmop/.

Source for this draft and an issue tracker can be found at https://github.com/WheaterW/draft-cui-nmop-auto-test.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 6 January 2027.

Copyright Notice

Table of Contents

1. Introduction 3

2. Problem Scope and Assumptions 4

3. Definitions and Acronyms 4

4. Background 5

5. Framework 5

5.1. Structured Protocol Representation 6

5.2. Coverage Scoping 7

5.3. Test Case Generation 8

5.4. Executable Artifact Generation, Execution, and Feedback 8

6. Design Considerations 9
6.1. Structured Workflow Boundaries 9
6.2. Test-Relevant Protocol Representation 9
6.3. Separation of Test Templates and Test Parameters 9
6.4. Human-in-the-Loop Review 9
6.5. Specification-Derived Testing 10
7. Use Cases 10
7.1. Update-Aware Testing 10
7.2. Specification-Derived Bug Exposure 10
7.3. Parameterized Boundary Testing 11
8. Operational Considerations and Open Issues 11
9. Conclusion 12
10. Security Considerations 12
11. IANA Considerations 12
12. Informative References 12
Acknowledgments 13
Contributors 13
Authors' Addresses 13

1. Introduction

Network protocol testing is used to validate whether an implementation behaves according to the semantics defined by protocol specifications. It is needed throughout the lifecycle of network systems, from implementation development to deployment and operational maintenance.

Traditional protocol testing remains largely manual. Engineers read long specifications, identify test points, design topologies, write test procedures, create DUT configurations and tester scripts, execute tests, and inspect results. Model-based approaches can automate parts of this process, but they often depend on protocol-specific models that are expensive to build and difficult to reuse across new or rapidly evolving protocols.

Recent advances in artificial intelligence, especially Large Language Models (LLMs) and AI agents, create new opportunities for automating parts of this workflow. However, protocol testing is not a generic text-to-code task. Generated tests need to preserve protocol semantics from the specification, reflect the intended coverage scope, coordinate tester and DUT behavior, execute in a controlled environment, and provide reviewable result checks.

This document therefore focuses on a framework question: how can a testing workflow carry test-relevant information from protocol specifications to executable test artifacts while remaining auditable and controllable? The framework decomposes the workflow into structured protocol representation, coverage scoping, test case generation, executable artifact generation, test execution, and feedback-based refinement. At each stage boundary, the framework preserves not only the generated artifact, but also the protocol semantics, assumptions, and review points needed by later stages.

2. Problem Scope and Assumptions

This document focuses on specification-derived protocol testing. The primary input is a protocol specification, such as an RFC document. The target outputs are test cases, tester scripts, DUT configurations, execution results, and reports that can be traced back to the specification.

The framework is mainly intended for conformance, functional, robustness, regression, and related protocol-behavior tests. It does not attempt to cover all implementation-specific or vendor-specific behavior, which may require additional inputs beyond protocol specifications.

The framework does not assume a fixed level of capability for AI-assisted components. As these capabilities evolve, the degree of automation and the placement of human review can be adjusted according to the risk and maturity of each workflow stage. The intended role of automation is to reduce repetitive expert effort while preserving traceability and expert control.

3. Definitions and Acronyms

DUT: Device Under Test

Tester: A device with sufficient network protocol functionality and test-control capabilities to execute test cases. It can generate and receive test-specific packets or traffic, emulate target network behaviors, collect observations, and analyze results.

LLM: Large Language Model

AI Agent: A system that can assist with or drive parts of multi-step workflows by using tools and making decisions based on feedback, subject to configured constraints and review points.

API: Application Programming Interface

CLI: Command Line Interface

Test Case: A specification of conditions and inputs to evaluate a protocol behavior.

Tester Script: An executable program or sequence of instructions that controls a tester during test execution.

4. Background

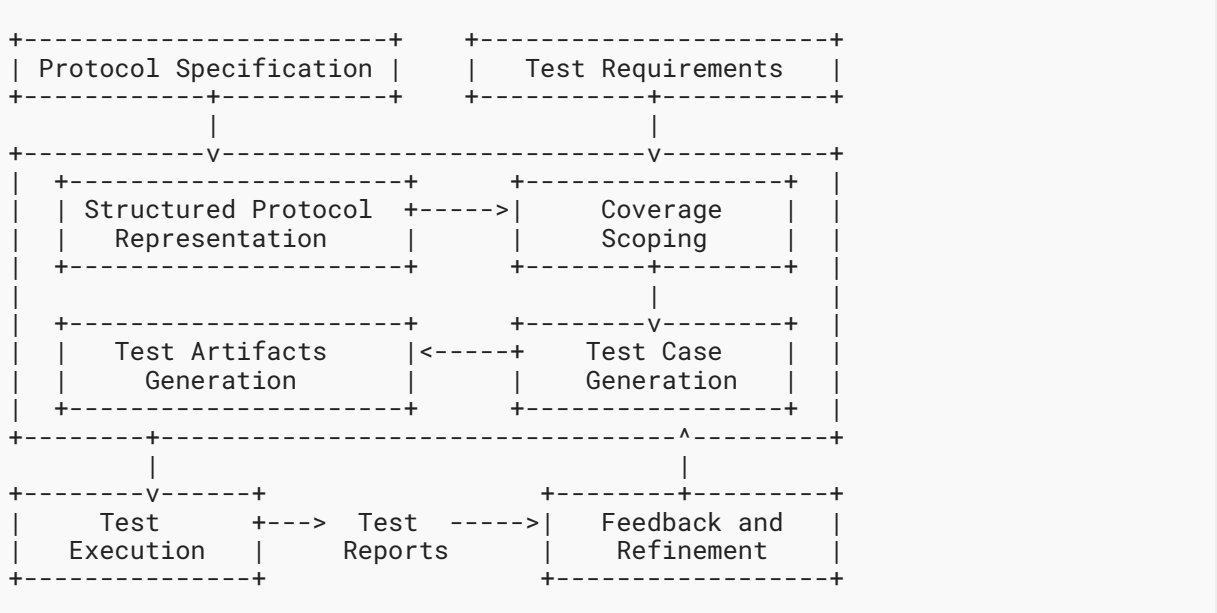
Protocol testing is required during device development, where vendors verify that their implementations conform to protocol specifications, and during procurement evaluation, where third-party organizations perform black-box conformance testing against a neutral standard. Both scenarios require coordinated test cases, DUT configuration, tester behavior, execution observations, and result evaluation.

A DUT can be a physical network device such as a switch, router, or firewall, or a virtual network device such as an FRRouting (FRR) instance [FRRouting]. A tester is typically controlled by scripts that configure protocol behavior, generate test traffic, collect observations, and support result evaluation.

Before executing a test case, the DUT is initialized with test-specific configurations. During the test, tester actions and DUT configuration need to remain consistent with the intended topology, protocol parameters, procedure, and expected results.

5. Framework

The AI-assisted network protocol testing framework is illustrated in the figure below. Test requirements are an external input, provided by operators, testing teams, or certification teams for each test campaign.



The framework has six stages:

1. Structured protocol representation: transform specification text into a test-relevant representation.
2. Coverage scoping: reconcile the protocol representation with test requirements to produce an explicit, reviewable coverage scope.
3. Test case generation: derive test templates, parameters, and oracles from included scope items.
4. Test artifacts generation: translate test cases into coordinated tester scripts and DUT configurations.
5. Test execution: run the generated artifacts in a controlled test environment.
6. Feedback and refinement: analyze failures and decide whether to refine artifacts, scope, or bug reports.

Each stage produces intermediate outputs that can include generated content, source references, assumptions, constraints, and review status.

The six stages support two complementary cycles. In the forward cycle, coverage scoping and test case generation produce the test suite from the specification and the test requirements. In the backward cycle, execution feedback is analyzed to localize the likely source of a failure or coverage gap, starting from concrete execution artifacts and moving toward more abstract scope or representation issues when needed. An AI agent can assist in both cycles, with human review applied where automation uncertainty or risk is high, or where automation maturity is insufficient.

5.1. Structured Protocol Representation

Protocol specifications are usually written for human readers. They include message formats, state machines, timers, variables, algorithms, exception handling, and normative behavior spread across sections and sometimes across multiple update documents. Directly generating executable tests from such text risks losing details that determine whether a test is valid.

The framework therefore introduces a structured protocol representation as an intermediate artifact between specification text and downstream test generation. The representation is not required to be a complete formal model of the protocol. Instead, it preserves the protocol semantics that are relevant for testing.

Such a representation can include:

- message formats, fields, constraints, and valid or invalid values
- local data structures, timers, counters, and protocol variables
- state machines and state transitions
- event-action rules and packet processing behavior
- protocol algorithms, such as route selection or path computation
- error handling and exception behavior

- relationships among modules, such as which messages trigger which transitions or algorithms
- links back to source specification sections

A practical construction workflow can be organized into three steps.

First, specification analysis identifies sections, cross-references, normative statements, summaries, and update relationships in the specification. This step can combine rule-based extraction with AI-assisted summarization.

Second, module induction groups related text into protocol modules, such as message formats, state machines, algorithms, and event-action rules. Each module remains linked to the source text used to construct it.

Third, formalization serializes each module into a structured representation that can be queried, traversed, reviewed, and used by test generation tools.

Protocol updates require special handling. Update documents often add, modify, or deprecate specific protocol behavior rather than restating a complete protocol. An update-aware representation identifies update points, maps them to existing modules, and expresses the update as a differential change to the base representation.

The main design trade-off is between completeness and usefulness. A fully formal protocol model can be expensive to build and difficult to apply across many protocols. A testing-oriented representation instead focuses on the semantics needed to generate valid tests, derive oracles, and trace failures back to specifications.

5.2. Coverage Scoping

A structured protocol representation describes what a protocol specification defines. It does not determine which definitions a particular test campaign intends to cover. Coverage scoping uses the representation to select the protocol behaviors that a test suite will exercise and to document which items are excluded and why.

Coverage scoping takes two inputs: the structured protocol representation and the test requirements for a test campaign. These requirements are external to the framework and reflect the campaign's objectives, constraints, and priorities.

The output is a coverage scope: a structured decision record in which each referenced protocol behavior is marked as included or excluded, assigned a priority, and, when excluded, accompanied by a documented reason.

The coverage scope serves three purposes. First, it turns coverage into an inspectable plan: a human reviewer can assess whether the set of included and excluded behaviors is acceptable for the testing objective. Second, it provides the input to test case generation, so that generation focuses on how to test rather than what to test. Third, during iterative refinement, it provides a

controlled basis for deciding whether a newly observed behavior or coverage gap should be added to the campaign scope, remain excluded, or trigger a revision of the documented scoping rationale.

An AI agent can assist in producing an initial coverage scope by traversing the representation, applying the test requirements, and proposing inclusion or exclusion with documented reasoning. Human review can be applied as needed before the scope is used for test generation.

5.3. Test Case Generation

Once a coverage scope has been established, test generation derives test cases from the included scope items. These test cases can be based on normative statements, message constraints, state transitions, algorithms, error handling requirements, and update deltas in the representation.

The framework separates the reusable structure of a test from the concrete values used to instantiate it. A test template captures the reusable structure, including:

- test objective
- specification reference
- test topology
- preconditions and static configuration
- test procedure
- expected result or oracle
- variable placeholders

Parameters fill those placeholders with concrete values. Parameter generation can include valid values, invalid values, boundary values, timing values, topology variants, and values computed by helper logic. Oracle values can also require computation, especially when expected behavior depends on route selection, timers, path attributes, or other protocol algorithms.

This separation is a key design choice. Templates provide breadth across protocol functions and test scenarios. Parameters provide depth across value spaces and boundary conditions. Equivalence partitioning or similar reduction techniques can then group parameter combinations that exercise the same protocol behavior, reducing redundant execution without discarding meaningful coverage.

Generated test cases should remain reviewable. Review can consider whether a test reflects the intended protocol semantics and whether the resulting suite exercises the behaviors and parameter spaces identified in the coverage scope.

5.4. Executable Artifact Generation, Execution, and Feedback

Executable artifact generation translates abstract test cases into runnable tester scripts and DUT configurations. In practice, these artifacts are often synthesized from tester API documentation, DUT configuration manuals, prior scripts, and testbed context. Even when generated artifacts appear plausible, they can contain API misuse, invalid configuration, timing errors, or mismatches between tester behavior and DUT configuration.

The framework therefore treats artifact generation and test execution as a refinement loop. An AI agent can generate candidate artifacts, execute or dry-run them in a controlled test environment, collect execution feedback, and revise the artifacts before a result is treated as evidence about the DUT. Feedback can include syntax errors, API responses, device diagnostics, packet captures, and pass/fail observations.

Feedback analysis and refinement are not limited to executable artifacts. When execution feedback indicates that the artifacts correctly implement the intended test case but the test remains invalid, ambiguous, or inconclusive, feedback analysis can trace the problem back to the test case, the coverage scope, or the structured representation. Refinement actions can therefore update executable artifacts, oracle logic, test-case definitions, scoping decisions, or representation content. Human review can be applied as needed when the interpretation of feedback or the resulting refinement actions carry high risk.

6. Design Considerations

6.1. Structured Workflow Boundaries

Single-pass prompting can produce useful drafts, but protocol testing benefits from a more structured workflow. Decomposing the workflow into stages makes intermediate outputs reviewable and allows each stage to focus on a specific part of the testing problem, such as preserving protocol semantics, defining coverage, deriving test cases, generating executable artifacts, and interpreting feedback. This provides a more systematic and inspectable path from specification text to executable tests.

6.2. Test-Relevant Protocol Representation

Complete formal models can support rigorous analysis, but they are expensive to build and difficult to maintain across many protocols and update documents. A test-relevant representation makes a narrower claim: it preserves the semantics needed for testing rather than modeling every aspect of the protocol. This scoped representation is more feasible for broad use across protocol testing workflows.

6.3. Separation of Test Templates and Test Parameters

Generating many concrete tests directly can produce large and redundant suites. Separating test templates from test parameters allows broad functional coverage and deeper parameter exploration to scale independently. This separation also requires support for parameter reduction, oracle computation, and test-suite scheduling.

6.4. Human-in-the-Loop Review

AI-assisted workflows can help analyze execution feedback, but some interpretations and refinement actions can carry high risk. The framework therefore allows human review to be applied where automation uncertainty or risk is high, or where automation maturity is insufficient.

6.5. Specification-Derived Testing

Specification-derived testing targets behavior defined by protocol specifications. It can miss implementation-specific or vendor-specific behavior that requires inputs beyond the protocol specification. Such extensions are possible, but they require additional input modeling and are outside the primary scope of this framework.

7. Use Cases

This section uses routing-protocol testing examples to illustrate how the framework can support operational test workflows.

7.1. Update-Aware Testing

Protocol behavior changes over time as RFCs are updated. For example, RFC 9774 [RFC9774] deprecated the AS_SET path segment type in BGP, updating behavior originally specified in RFC 4271 [RFC4271]. A framework that treats the update document in isolation may miss the base-protocol context needed to derive an executable test.

An update-aware representation can record the RFC 9774 change as a delta to the BGP path-attribute behavior in the base representation. Test generation can then focus on the changed behavior, for example by constructing a route advertisement that contains AS_SET and checking whether the DUT handles it according to the updated specification.

This example illustrates why update documents are best represented as changes to existing protocol modules rather than as standalone specifications.

7.2. Specification-Derived Bug Exposure

Specifications can define behavior that is easy to overlook in manually curated test suites. For example, RFC 2453 [RFC2453] defines conditions under which a RIP router originates and advertises a default route to neighbors (Section 3.7). A manually curated test suite might verify basic route exchange without exercising this specific condition.

A structured representation can make the condition explicit. Test generation can then configure the DUT to originate a default route, observe RIP update messages, and check whether the expected 0.0.0.0/0 route is advertised. In an experimental test workflow, this test exposed a default-route handling defect in a deployed RIP implementation.

This example illustrates how specification-derived testing can expose coverage gaps in manually curated suites and exercise protocol behavior that is easy to miss in basic functional tests.

7.3. Parameterized Boundary Testing

Some bugs appear only under specific parameter relationships rather than under a single fixed input. In OSPF, for example, DR election depends on relative router priorities. A test template can capture the protocol scenario, while parameter instantiation explores priority relationships such as equal priorities and asymmetric priorities.

In an OSPF route-overwrite test case, the bug was triggered only when the tester-side peers used asymmetric priorities and a DR re-election was forced. The corresponding parameterized test distinguished this boundary condition from a symmetric-priority configuration that did not trigger the failure.

This example illustrates why test depth depends on parameter relationships and oracle computation, not only on the number of generated test cases.

8. Operational Considerations and Open Issues

Several operational considerations and open issues remain for applying AI-assisted protocol testing in operational test environments:

1. Assessing representation fidelity: A structured protocol representation is useful only if it preserves the semantics needed for testing. Operators and tool developers lack common criteria for assessing whether message formats, state transitions, algorithms, exception handling, and update effects have been captured with sufficient fidelity. This makes it difficult to determine when a representation is adequate for a testing objective.
2. Validating coverage scope: Coverage scope is a decision record rather than a property directly defined by the protocol specification. Operational use requires assessing whether the included and excluded behaviors are appropriate for a testing objective, and whether important protocol behaviors have been omitted. This is difficult because both specifications and test requirements are often expressed in natural language.
3. Generating and validating oracles: Many protocol tests depend on expected results that are derived from algorithms, timers, ordering constraints, or parameter relationships. Some oracles can be computed directly, while others require interpretation of protocol context and testbed behavior. Operational test environments need practical ways to generate such oracles and validate their correctness.
4. Making feedback-driven refinement reliable: Execution feedback can reveal artifact errors, invalid test cases, incorrect oracles, environment problems, or deviations from the protocol specification. When an AI agent participates in artifact generation and refinement, its own actions can also contribute to the failure. A challenge is to distinguish these causes reliably and record enough context for refinement decisions to be replayed and audited.

5. Balancing automation, review, and traceability: AI-assisted workflows need traceability from test results back to executable artifacts, test cases, coverage scope, protocol representation, and specification text. Operational deployments need to determine what context should be recorded, how review decisions should be represented, and where human review should be placed as automation capability and operational risk change.

9. Conclusion

This document has described a framework for AI-assisted network protocol testing from specifications. The framework provides a structured workflow, from protocol representation through coverage scoping, test generation, execution, and feedback, in which stage boundaries are explicit, intermediate outputs remain traceable to the specification, and human review can be applied where automation uncertainty or risk is high. The framework identifies design considerations and open issues for making AI-assisted protocol testing more systematic, auditable, and controllable in operational test environments.

10. Security Considerations

1. Execution of generated artifacts: Automatically generated tester scripts and DUT configurations can misconfigure devices, generate unintended traffic, or disrupt a test environment. Such artifacts should be executed only in isolated and recoverable test environments, with appropriate access controls, rollback mechanisms, and traffic containment. Syntax checks, dry runs, and semantic review can reduce risk, but they do not eliminate the need for operational safeguards.
2. AI-assisted generation and refinement: LLMs and AI agents may produce incorrect outputs due to model limitations, incomplete context, or hallucination. Such errors may lead to false positives, where a valid DUT is reported as failing, or false negatives, where a real deviation from the protocol specification is missed. Systems using such components should treat generated actions and conclusions as untrusted until they are checked against the test objective, the generated artifacts, and the execution evidence.
3. Exposure of sensitive testbed information: DUT configurations, tester scripts, logs, packet captures, and telemetry can contain sensitive operational or implementation information. Such inputs and outputs should be protected according to the confidentiality requirements of the test environment.

11. IANA Considerations

This document has no IANA actions.

12. Informative References

[FRRouting] "FRRouting", n.d., <https://frrouting.org/>.

[RFC2453] Malkin, G., "RIP Version 2", STD 56, RFC 2453, DOI 10.17487/RFC2453, November 1998, <https://www.rfc-editor.org/rfc/rfc2453>.

[RFC4271] Rekhter, Y., Ed., Li, T., Ed., and S. Hares, Ed., "A Border Gateway Protocol 4 (BGP-4)", RFC 4271, DOI 10.17487/RFC4271, January 2006, <https://www.rfc-editor.org/rfc/rfc4271>.

[RFC9774] Kumari, W., Sriram, K., Hannachi, L., and J. Haas, "Deprecation of AS_SET and AS_CONFED_SET in BGP", RFC 9774, DOI 10.17487/RFC9774, May 2025, <https://www.rfc-editor.org/rfc/rfc9774>.

Acknowledgments

This work is supported by the National Key R&D Program of China.

Contributors

Zhen Li
Beijing Xinertel Technology Co., Ltd.
Email: lizhen_fz@xinertel.com

Zhanyou Li
Beijing Xinertel Technology Co., Ltd.
Email: lizy@xinertel.com

Authors' Addresses

Yong Cui
Tsinghua University
Email: cuiyong@tsinghua.edu.cn

Yunze Wei
Tsinghua University
Email: wyz23@mails.tsinghua.edu.cn

Kaiwen Chi
Tsinghua University
Email: ckw24@mails.tsinghua.edu.cn

Xiaohui Xie
Tsinghua University
Email: xiexiaohui@tsinghua.edu.cn

Shailesh Prabhu
Nokia
Email: shailesh.prabhu@nokia.com